🏆 Guide Complet & Documentation Technique
Projet Data Analyse : CAN 2025 (Maroc)

Architecture End-to-End, Méthodologie Étape par Étape & Codes Sources Complets d'Extraction Web (FotMob, StatsBomb, API-Football, SQL, Python & Streamlit)

Auteur : Amine | **Dossier Projet :** `C:\Users\Amine\Desktop\can2025-data-analysis`

1. Web Scraping FotMob (__NEXT_DATA__)
2. APIs REST (StatsBomb & API-Football)
3. Python ETL (Pandas & Multi-threading)
4. Base Relationnelle SQL (SQLite3)
5. Application Web (Streamlit & Plotly)
6. BI (Excel & Power BI DAX)

1. Vue d'Ensemble & Architecture du Projet

Ce projet portfolio a été conçu pour couvrir **100 % des compétences attendues chez un Data Analyst** en entreprise : de la collecte brute sur le Web (scraping et APIs) jusqu'à la restitution interactive pour la prise de décision sportive (Scouting, Analyse Tactique et Bilan de Compétition).

Étape du Pipeline	Outils & Technologies	Fichiers du Projet	Résultat Obtenu
Étape 1 : Collecte Web & APIs	Python (`urllib` , `re` , `json` , `ThreadPoolExecutor`)	`scripts/04_scrape_fotmob_can2025.py` `scripts/05_scrape_fotmob_match_details.py` `scripts/03_fetch_real_data.py`	Extraction réelle de **52 matchs**, **1 026 tirs géolocalisés (xG)**, **710 événements** et **2 445 fiches joueurs** depuis FotMob.
Étape 2 : Nettoyage	Python (`pandas`) & Excel	`scripts/01_clean_and_build_db.py` `guides/01_guide_excel.md`	Suppression des doublons,

Étape du Pipeline	Outils & Technologies	Fichiers du Projet	Résultat Obtenu
& ETL			nettoyage des chaînes, imputation médiane et calcul des ratios par 90 minutes.
Étape 3 : Base de Données SQL	SQLite3 (`SQL`)	`data/can2025.db` `sql/schema.sql` `sql/business_queries.sql`	9 tables relationnelles interrogeables avec `JOIN` , `CTE (WITH)` et Window Functions (`DENSE_RANK`).
Étape 4 : Visualisation & App Web	Python (`streamlit` , `plotly` , `matplotlib`)	`app/streamlit_app.py` `scripts/02_generate_charts.py`	Plateforme interactive à 6 onglets (Diagnostic 24 équipes, Shotmap sur demi-terrain, Match Center, Scouting) + 5 graphiques PNG HD.
Étape 5 : Business Intelligence	Power BI / Tableau & Excel	`guides/02_guide_powerbi_tableau.md`	Modèle en étoile et 6 mesures DAX prêtes pour la création de rapports exécutifs.

> 💡 **Comment expliquer cette démarche en entretien :**
> *« Plutôt que de télécharger un simple fichier CSV statique, j'ai construit un pipeline automatisé en Python capable d'extraire directement les données structurées de FotMob (League 289) et de StatsBomb Open Data, de les nettoyer avec Pandas, de les stocker dans une base SQL relationnelle (SQLite), puis de les exposer dans une application interactive Streamlit incluant une carte des tirs (Shotmap) et un explorateur de feuilles de match. »*

2. Étape 1 : Extraction Globale de la CAN 2025 depuis FotMob

Comment fonctionne l'extraction sur FotMob ?

Le site `https://www.fotmob.com/fr/leagues/289/overview/africa-cup-nations` est construit avec le framework **Next.js**. Lorsqu'on charge la page HTML, toutes les données structurées du tournoi sont injectées par le serveur dans une balise cachée : `<script id="__NEXT_DATA__" type="application/json">...</script>`.

Notre premier script (`scripts/04_scrape_fotmob_can2025.py`) utilise une expression régulière pour isoler ce bloc JSON, puis télécharge également les sous-fichiers JSON de statistiques détaillées (`https://data.fotmob.com/stats/289/season/24309/*.json`). Il extrait :

- **Les 52 matchs** avec leur identifiant unique FotMob (`fotmob_match_id`) et l'URL de chaque feuille de match.
- **Les classements officiels des 6 groupes (A à F)** et le tableau des meilleurs 3es (30 entrées).
- **Les 123 classements de statistiques individuelles** (Buteurs, Passeurs, Notes FotMob, Expected Goals xG, Occasions créées...).
- **Les 84 classements de statistiques collectives par équipe**.

Code Source Complet : `scripts/04_scrape_fotmob_can2025.py`

```
# -*- coding: utf-8 -*-
"""
Extraction Complète de la CAN 2025 depuis FotMob (League ID = 289)
URL cible : https://www.fotmob.com/fr/leagues/289/overview/africa-cup-nations
Fichier   : scripts/04_scrape_fotmob_can2025.py

Ce script utilise une double stratégie robuste :
1. Interrogation de l'API JSON interne de FotMob (/api/leagues?id=289)
2. Extraction automatique du bloc Next.js (<script id="__NEXT_DATA__">) depuis la page HTML
   https://www.fotmob.com/fr/leagues/289/overview/africa-cup-nations
3. Téléchargement des sous-fichiers JSON de statistiques détaillées (Joueurs & Équipes)
4. Export en CSV dans data/raw/ et sauvegarde dans la base SQLite data/can2025.db
"""

import os
import re
import sys
import json
import sqlite3
import urllib.request
import urllib.parse
import pandas as pd

if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
RAW_DIR = os.path.join(BASE_DIR, "data", "raw")
DB_PATH = os.path.join(BASE_DIR, "data", "can2025.db")
os.makedirs(RAW_DIR, exist_ok=True)

FOTMOB_OVERVIEW_URL = "https://www.fotmob.com/fr/leagues/289/overview/africa-cup-nations"
```

```
FOTMOB_API_URL = "https://www.fotmob.com/api/leagues?id=289"

HEADERS = {
    "User-Agent": (
        "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
        "AppleWebKit/537.36 (KHTML, like Gecko) "
        "Chrome/124.0.0.0 Safari/537.36"
    ),
    "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,application/json;q=0.8,*/*;q=0.7",
    "Accept-Language": "fr-FR,fr;q=0.9,en-US;q=0.8,en;q=0.7",
    "Referer": "https://www.fotmob.com/fr/leagues/289/overview/africa-cup-nations",
}


def fetch_url_text(url):
    """Télécharge le contenu texte (HTML ou JSON) d'une URL FotMob."""
    req = urllib.request.Request(url, headers=HEADERS)
    with urllib.request.urlopen(req, timeout=20) as resp:
        return resp.read().decode("utf-8", errors="replace")


def get_fotmob_league_payload():
    """
    Récupère les données complètes de la ligue 289 (Africa Cup of Nations)
    soit via l'API directe, soit via la balise <script id="__NEXT_DATA__"> de la page HTML.
    """
    # Méthode 1 : Extraction depuis la page HTML officielle demandée (__NEXT_DATA__)
    print(f"🌐 1. Connexion à la page : {FOTMOB_OVERVIEW_URL}")
    try:
        html = fetch_url_text(FOTMOB_OVERVIEW_URL)
        match = re.search(
            r'<script id="__NEXT_DATA__" type="application/json"[^>]*>(.*?)</script>',
            html,
            re.DOTALL
        )
        if match:
            next_data = json.loads(match.group(1))
            page_props = next_data.get("props", {}).get("pageProps", {})
            # Dans FotMob Next.js, les données de la ligue sont soit directement dans pageProps,
            # soit dans pageProps["fallback"] / pageProps["initialState"]
            if "matches" in page_props or "table" in page_props or "details" in page_props:
                print("✅ Données extraites avec succès depuis le bloc __NEXT_DATA__ (pageProps) !")
                return page_props
            fallback = page_props.get("fallback", {})
            for k, val in fallback.items():
                if isinstance(val, dict) and ("matches" in val or "table" in val or "details" in val):
                    print(f"✅ Données extraites avec succès depuis __NEXT_DATA__ (fallback['{k}']) !")
                    return val
    except Exception as e:
        print(f"ℹ️ Passage à la méthode API directe ({e})")

    # Méthode 2 : Interrogation de l'endpoint JSON /api/leagues?id=289
    print(f"🌐 2. Interrogation de l'API FotMob : {FOTMOB_API_URL}")
    try:
        raw_json = fetch_url_text(FOTMOB_API_URL)
        data = json.loads(raw_json)
        print("✅ Données extraites avec succès depuis /api/leagues?id=289 !")
        return data
    except Exception as e:
        print(f"⚠️ Erreur lors de l'accès à l'API FotMob : {e}")
        return None


def parse_matches(payload):
    """Extrait tous les matchs de la CAN depuis le JSON FotMob."""
```

```
    matches_root = payload.get("matches", {})
    all_matches = (
        matches_root.get("allMatches")
        or matches_root.get("data", {}).get("allMatches")
        or payload.get("fixtures", {}).get("allMatches")
        or []
    )

    rows = []
    for m in all_matches:
        home = m.get("home", {})
        away = m.get("away", {})
        status = m.get("status", {})
        score_str = status.get("scoreStr", "")
        reason = status.get("reason", {}).get("short", "")

        rows.append({
            "fotmob_match_id": m.get("id"),
            "round": m.get("roundName") or m.get("round"),
            "date_utc": status.get("utcTime"),
            "home_team_id": home.get("id"),
            "home_team": home.get("name"),
            "away_team_id": away.get("id"),
            "away_team": away.get("name"),
            "home_score": home.get("score"),
            "away_score": away.get("score"),
            "score_str": score_str,
            "status_note": reason,
            "finished": status.get("finished"),
            "cancelled": status.get("cancelled"),
            "match_url": "https://www.fotmob.com" + str(m.get("pageUrl", ""))
        })

    return pd.DataFrame(rows)


def parse_standings(payload):
    """Extrait le classement des 6 groupes (Groupes A à F) depuis le JSON FotMob."""
    table_list = payload.get("table", [])
    rows = []

    for tbl_block in table_list:
        data = tbl_block.get("data", tbl_block)
        # Tournoi à plusieurs groupes (tables list)
        tables = data.get("tables", [data])
        for grp in tables:
            group_name = grp.get("leagueName") or grp.get("GroupName") or "Groupe"
            for team in grp.get("table", {}).get("all", []):
                scores_str = str(team.get("scoresStr", "0-0"))
                gf, ga = (scores_str.split("-") + ["0", "0"])[:2]
                rows.append({
                    "group_name": group_name,
                    "rank": team.get("idx"),
                    "team_id": team.get("id"),
                    "team_name": team.get("name"),
                    "played": team.get("played"),
                    "wins": team.get("wins"),
                    "draws": team.get("draws"),
                    "losses": team.get("losses"),
                    "goals_for": int(gf.strip()) if gf.strip().isdigit() else 0,
                    "goals_against": int(ga.strip()) if ga.strip().isdigit() else 0,
                    "goal_diff": team.get("goalConDiff"),
                    "points": team.get("pts"),
                })
```

```
    return pd.DataFrame(rows)


def parse_stats_categories(payload, kind="players"):
    """
    Extrait les statistiques de joueurs (buts, passes D, notes FotMob, xG...)
    ou d'équipes depuis la section 'stats' du JSON FotMob.
    """
    stats_root = payload.get("stats", {})
    categories = stats_root.get(kind, [])
    rows = []

    for cat in categories:
        cat_name = cat.get("header") or cat.get("name") or cat.get("title")
        fetch_url = cat.get("fetchAllUrl")

        # 1. Récupérer le Top 3 directement présent dans l'aperçu
        top_three = cat.get("topThree", [])
        for item in top_three:
            rows.append({
                "category": cat_name,
                "participant_id": item.get("id"),
                "name": item.get("name"),
                "team_id": item.get("teamId"),
                "team_name": item.get("teamName"),
                "stat_value": item.get("value"),
                "rank": item.get("rank"),
                "source_url": fetch_url
            })

        # 2. Si une URL JSON complète (fetchAllUrl) est fournie, tenter de récupérer toute la liste
        if fetch_url and fetch_url.startswith("http"):
            try:
                sub_json = json.loads(fetch_url_text(fetch_url))
                top_lists = sub_json.get("TopLists", [])
                for tl in top_lists:
                    for item in tl.get("StatList", []):
                        rows.append({
                            "category": cat_name,
                            "participant_id": item.get("ParticiantId") or item.get("ParticipantId"),
                            "name": item.get("ParticipantName"),
                            "team_id": item.get("TeamId"),
                            "team_name": item.get("TeamName"),
                            "stat_value": item.get("StatValue"),
                            "rank": item.get("Rank"),
                            "source_url": fetch_url
                        })
            except Exception:
                pass

    df = pd.DataFrame(rows)
    if not df.empty:
        df = df.drop_duplicates(subset=["category", "name", "team_name"], keep="last")
    return df


def main():
    print("=" * 70)
    print("🦁 EXTRACTION COMPLÈTE FOTMOB — CAN 2025 (LEAGUE ID = 289)")
    print("=" * 70 + "\n")

    payload = get_fotmob_league_payload()
    if not payload:
        print("❌ Impossible de récupérer le payload FotMob.")
        return
```

```
    # Sauvegarde du JSON brut complet pour archivage et exploration
    raw_json_path = os.path.join(RAW_DIR, "fotmob_can2025_raw.json")
    try:
        with open(raw_json_path, "w", encoding="utf-8") as f:
            json.dump(payload, f, ensure_ascii=False, indent=2)
        print(f"💾 JSON brut complet sauvegardé : {raw_json_path}")
    except Exception as e:
        print(f"⚠️ Impossible d'écrire le JSON brut : {e}")

    details = payload.get("details", {})
    print(f"📌 Compétition : {details.get('name', 'Africa Cup of Nations')} | Saison :
{details.get('selectedSeason', '2025')}")

    conn = sqlite3.connect(DB_PATH)

    # 1. Extraction des Matchs
    df_matches = parse_matches(payload)
    if not df_matches.empty:
        csv_m = os.path.join(RAW_DIR, "fotmob_matches_can2025.csv")
        df_matches.to_csv(csv_m, index=False, encoding="utf-8")
        df_matches.to_sql("fotmob_matches", conn, if_exists="replace", index=False)
        print(f"✅ Matchs extraits : {len(df_matches)} matchs -> {csv_m}")
        print(df_matches[["date_utc", "round", "home_team", "score_str",
"away_team"]].tail(8).to_string(index=False))
    else:
        print("ℹ️ Aucun match trouvé dans la clé 'matches' du payload.")

    # 2. Extraction des Classements des Groupes (Standings)
    df_standings = parse_standings(payload)
    if not df_standings.empty:
        csv_s = os.path.join(RAW_DIR, "fotmob_standings_can2025.csv")
        df_standings.to_csv(csv_s, index=False, encoding="utf-8")
        df_standings.to_sql("fotmob_standings", conn, if_exists="replace", index=False)
        print(f"\n✅ Classements des groupes extraits : {len(df_standings)} équipes -> {csv_s}")
        print(df_standings.head(8).to_string(index=False))

    # 3. Extraction des Statistiques Joueurs (Top Buteurs, Passes D, Notes FotMob, xG...)
    df_pstats = parse_stats_categories(payload, kind="players")
    if not df_pstats.empty:
        csv_p = os.path.join(RAW_DIR, "fotmob_player_stats_can2025.csv")
        df_pstats.to_csv(csv_p, index=False, encoding="utf-8")
        df_pstats.to_sql("fotmob_player_stats", conn, if_exists="replace", index=False)
        print(f"\n✅ Statistiques Joueurs extraites : {len(df_pstats)} entrées -> {csv_p}")
        print(df_pstats.head(10).to_string(index=False))

    # 4. Extraction des Statistiques Équipes
    df_tstats = parse_stats_categories(payload, kind="teams")
    if not df_tstats.empty:
        csv_t = os.path.join(RAW_DIR, "fotmob_team_stats_can2025.csv")
        df_tstats.to_csv(csv_t, index=False, encoding="utf-8")
        df_tstats.to_sql("fotmob_team_stats", conn, if_exists="replace", index=False)
        print(f"\n✅ Statistiques Équipes extraites : {len(df_tstats)} entrées -> {csv_t}")

    conn.close()
    print("\n🎉 Extraction FotMob terminée et synchronisée dans data/can2025.db !")


if __name__ == "__main__":
    main()
```

3. Étape 2 : Extraction Granulaire des 52 Feuilles de Match (Shotmap, Événements & Notes)

Pourquoi un second script multi-threadé ?

La page d'accueil de la compétition donne les scores finaux, mais pas les détails à l'intérieur de chaque match. Grâce aux 52 identifiants `fotmob_match_id` récupérés à l'étape précédente, le script `scripts/05_scrape_fotmob_match_details.py` utilise un `ThreadPoolExecutor(max_workers=8)` pour inspecter les **52 feuilles de match en parallèle** en moins d'une minute.

Pour chacun des 52 matchs, il extrait 5 tables :

1. `fotmob_matches_metadata` (**52 lignes**) : Stade, ville, affluence officielle, arbitre et Homme du match (*Player of the Match*).
2. `fotmob_match_stats` (**2 039 lignes**) : Toutes les statistiques collectives face-à-face par match.
3. `fotmob_match_events` (**710 lignes**) : Chaque but, passe décisive, carton jaune/rouge et remplacement minute par minute.
4. `fotmob_shotmap` (**1 026 tirs**) : Coordonnées exactes `(x, y)` sur le terrain (105x68), `xG`, `xGOT`, pied/tête et situation de jeu.
5. `fotmob_player_match_stats` (**2 445 lignes**) : Les prestations et notes FotMob des joueurs match par match.

Code Source Complet : `scripts/05_scrape_fotmob_match_details.py`

```
# -*- coding: utf-8 -*-
"""
Extraction Granulaire des 52 Matchs de la CAN 2025 depuis FotMob
Fichier : scripts/05_scrape_fotmob_match_details.py

À partir des 52 matchs extraits dans 'fotmob_matches_can2025.csv', ce script récupère
pour CHAQUE match (via le bloc __NEXT_DATA__ de la page du match ou /api/matchDetails) :
1. Les statistiques complètes du match (Possession, xG, Tirs, Grosses occasions, Passes, Duels...)
2. Les événements minute par minute (Buts, Passeurs, Cartons, Remplacements, Tirs au but)
3. La Shotmap complète (Chaque tir avec ses coordonnées X/Y sur le terrain, xG, xGOT, pied/tête)
4. Les compositions & notes individuelles des joueurs match par match
"""

import os
import re
import sys
import json
import sqlite3
import urllib.request
from concurrent.futures import ThreadPoolExecutor, as_completed
import pandas as pd

if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
RAW_DIR = os.path.join(BASE_DIR, "data", "raw")
DB_PATH = os.path.join(BASE_DIR, "data", "can2025.db")
```

```
HEADERS = {
    "User-Agent": (
        "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
        "AppleWebKit/537.36 (KHTML, like Gecko) "
        "Chrome/124.0.0.0 Safari/537.36"
    ),
    "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,application/json;q=0.8,*/*;q=0.7",
    "Accept-Language": "fr-FR,fr;q=0.9,en-US;q=0.8,en;q=0.7",
}


def fetch_single_match_payload(match_row):
    """
    Récupère le JSON détaillé d'un match FotMob à partir de son URL (via __NEXT_DATA__)
    ou de l'endpoint /api/matchDetails?matchId=...
    """
    match_id = str(match_row["fotmob_match_id"])
    raw_url = str(match_row["match_url"]).split("#")[0]
    # Ajouter ?id=match_id garantit que Next.js charge exactement ce match (et non un ancien H2H)
    page_url = f"{raw_url}?id={match_id}"

    # 1. Essai via la page HTML du match (__NEXT_DATA__)
    try:
        req = urllib.request.Request(page_url, headers=HEADERS)
        with urllib.request.urlopen(req, timeout=20) as resp:
            html = resp.read().decode("utf-8", errors="replace")
        m = re.search(
            r'<script id="__NEXT_DATA__" type="application/json"[^>]*>(.*?)</script>',
            html,
            re.DOTALL
        )
        if m:
            next_data = json.loads(m.group(1))
            page_props = next_data.get("props", {}).get("pageProps", {})
            if "content" in page_props or "general" in page_props:
                return match_id, page_props
            fallback = page_props.get("fallback", {})
            for k, val in fallback.items():
                if isinstance(val, dict) and ("content" in val or "general" in val):
                    return match_id, val
    except Exception:
        pass

    # 2. Essai via l'API directe /api/matchDetails
    try:
        api_url = f"https://www.fotmob.com/api/matchDetails?matchId={match_id}"
        req = urllib.request.Request(api_url, headers=HEADERS)
        with urllib.request.urlopen(req, timeout=15) as resp:
            data = json.loads(resp.read().decode("utf-8", errors="replace"))
            if "content" in data or "general" in data:
                return match_id, data
    except Exception:
        pass

    return match_id, None


def extract_from_match_payload(match_id, payload, meta_row):
    """Extrait les 4 listes (stats_equipes, evenements, tirs_shotmap, joueurs_match) d'un match."""
    general = payload.get("general") or {}
    header = payload.get("header") or {}
    content = payload.get("content") or {}

    home_team = meta_row["home_team"]
    away_team = meta_row["away_team"]
```

```
    round_name = meta_row["round"]
    match_date = str(meta_row["date_utc"])[:10]

    # Informations générales du match (Stade, Ville, Affluence, Arbitre)
    match_facts = content.get("matchFacts") or {}
    info_box = match_facts.get("infoBox") or {}
    stadium_info = info_box.get("Stadium") if isinstance(info_box.get("Stadium"), dict) else {}
    attendance = info_box.get("Attendance")
    referee_info = info_box.get("Referee") if isinstance(info_box.get("Referee"), dict) else {}
    potm_name = ((match_facts.get("playerOfTheMatch") or {}).get("name") or {})
    if isinstance(potm_name, dict):
        potm_full = potm_name.get("fullName")
    else:
        potm_full = str(potm_name) if potm_name else None

    meta_info = {
        "fotmob_match_id": match_id,
        "match_date": match_date,
        "round": round_name,
        "home_team": home_team,
        "away_team": away_team,
        "score_str": meta_row["score_str"],
        "stadium_name": (stadium_info or {}).get("name"),
        "city": (stadium_info or {}).get("city"),
        "attendance": attendance,
        "referee": (referee_info or {}).get("text"),
        "player_of_the_match": potm_full
    }

    # ------------------------------------------------------------------------
    # A. STATISTIQUES COLLECTIVES DU MATCH (content -> stats)
    # ------------------------------------------------------------------------
    stats_rows = []
    stats_root = content.get("stats") or {}
    periods = stats_root.get("Periods") or {}
    all_period = periods.get("All") or stats_root.get("Ede") or {}
    stat_groups = (all_period.get("stats") or []) if isinstance(all_period, dict) else []

    for grp in stat_groups:
        if not isinstance(grp, dict):
            continue
        grp_title = grp.get("title")
        for st_item in (grp.get("stats") or []):
            if not isinstance(st_item, dict) or st_item.get("type") == "title":
                continue
            vals = st_item.get("stats") or [None, None]
            if isinstance(vals, list) and len(vals) >= 2:
                stats_rows.append({
                    "fotmob_match_id": match_id,
                    "match_date": match_date,
                    "home_team": home_team,
                    "away_team": away_team,
                    "stat_group": grp_title,
                    "stat_name": st_item.get("title"),
                    "stat_key": st_item.get("key"),
                    "home_value": str(vals[0]),
                    "away_value": str(vals[1]),
                })

    # ------------------------------------------------------------------------
    # B. ÉVÉNEMENTS MINUTE PAR MINUTE (Buts, Cartons, Remplacements)
    # ------------------------------------------------------------------------
    event_rows = []
    events_list = ((match_facts.get("events") or {}).get("events")) or []
    for ev in events_list:
```

```
        if not isinstance(ev, dict):
            continue
        ev_type = ev.get("type")
        if ev_type not in ("Goal", "Card", "Substitution", "MissedPenalty"):
            continue
        is_home = ev.get("isHome")
        player_obj = ev.get("player") if isinstance(ev.get("player"), dict) else {}
        player_name = player_obj.get("name") or ev.get("nameStr")
        swap_list = ev.get("swap") if isinstance(ev.get("swap"), list) and len(ev.get("swap")) > 0 else []
        swap_name = swap_list[-1].get("name") if swap_list and isinstance(swap_list[-1], dict) else None
        assist_str = ev.get("assistStr") or swap_name or ev.get("assistInput") or ""
        new_score = ev.get("newScore") if isinstance(ev.get("newScore"), list) and len(ev.get("newScore")) >=
2 else [None, None]
        event_rows.append({
            "fotmob_match_id": match_id,
            "match_date": match_date,
            "home_team": home_team,
            "away_team": away_team,
            "team_side": home_team if is_home else away_team,
            "minute": ev.get("time"),
            "overload_minute": ev.get("overloadTime"),
            "event_type": ev_type,
            "card_type": ev.get("card"),
            "player_name": player_name,
            "detail": str(assist_str),
            "home_score_after": new_score[0],
            "away_score_after": new_score[1],
        })

    # ------------------------------------------------------------------------
    # C. SHOTMAP COMPLÈTE (Coordonnées X/Y des tirs, xG, xGOT, résultat)
    # ------------------------------------------------------------------------
    shot_rows = []
    shots_list = ((content.get("shotmap") or {}).get("shots")) or []
    for sh in shots_list:
        if not isinstance(sh, dict):
            continue
        shot_rows.append({
            "fotmob_match_id": match_id,
            "match_date": match_date,
            "home_team": home_team,
            "away_team": away_team,
            "team_id": sh.get("teamId"),
            "player_id": sh.get("playerId"),
            "player_name": sh.get("playerName"),
            "minute": sh.get("min"),
            "event_type": sh.get("eventType"),     # Goal, AttemptSaved, Miss, Post
            "situation": sh.get("situation"),       # RegularPlay, Penalty, SetPiece, FastBreak
            "shot_type": sh.get("shotType"),        # RightFoot, LeftFoot, Header
            "x": sh.get("x"),
            "y": sh.get("y"),
            "xg": sh.get("expectedGoals"),
            "xgot": sh.get("expectedGoalsOnTarget"),
            "is_on_target": sh.get("isOnTarget"),
            "is_blocked": sh.get("isBlocked"),
            "is_own_goal": sh.get("isOwnGoal"),
        })

    # ------------------------------------------------------------------------
    # D. STATISTIQUES INDIVIDUELLES DES JOUEURS SUR LE MATCH (playerStats)
    # ------------------------------------------------------------------------
    player_match_rows = []
    pstats_dict = content.get("playerStats") or {}
    if isinstance(pstats_dict, dict):
        for pid, pinfo in pstats_dict.items():
```

```
            if not isinstance(pinfo, dict):
                continue
            pname = pinfo.get("name")
            tname = pinfo.get("teamName")
            is_goalkeeper = pinfo.get("isGoalkeeper")

            flat_metrics = {}
            for section in (pinfo.get("stats") or []):
                if not isinstance(section, dict):
                    continue
                sec_stats = section.get("stats") or {}
                if isinstance(sec_stats, dict):
                    for s_label, s_obj in sec_stats.items():
                        if isinstance(s_obj, dict):
                            stat_inner = s_obj.get("stat") if isinstance(s_obj.get("stat"), dict) else {}
                            val = stat_inner.get("value")
                            flat_metrics[s_obj.get("key") or s_label] = val

            player_match_rows.append({
                "fotmob_match_id": match_id,
                "match_date": match_date,
                "player_id": pid,
                "player_name": pname,
                "team_name": tname,
                "is_goalkeeper": is_goalkeeper,
                "fotmob_rating": flat_metrics.get("rating_title"),
                "minutes_played": flat_metrics.get("minutes_played"),
                "goals": flat_metrics.get("goals"),
                "assists": flat_metrics.get("assists"),
                "xg": flat_metrics.get("expected_goals"),
                "xa": flat_metrics.get("expected_assists"),
                "total_shots": flat_metrics.get("total_shots"),
                "chances_created": flat_metrics.get("chances_created"),
                "touches": flat_metrics.get("touches"),
                "accurate_passes": flat_metrics.get("accurate_passes"),
                "tackles_succeeded": flat_metrics.get("Matchstats.headers.tackles"),
                "interceptions": flat_metrics.get("interceptions"),
                "duels_won": flat_metrics.get("duel_won"),
            })

    return meta_info, stats_rows, event_rows, shot_rows, player_match_rows


def main():
    print("=" * 80)
    print("🏟️ EXTRACTION DÉTAILLÉE DES 52 MATCHS FOTMOB — CAN 2025")
    print("=" * 80 + "\n")

    matches_csv = os.path.join(RAW_DIR, "fotmob_matches_can2025.csv")
    if not os.path.exists(matches_csv):
        print("❌ Fichier fotmob_matches_can2025.csv introuvable. Lancez d'abord
04_scrape_fotmob_can2025.py")
        return

    df_matches = pd.read_csv(matches_csv)
    print(f"📋 {len(df_matches)} matchs à extraire en parallèle (ThreadPoolExecutor)...\n")

    all_meta = []
    all_stats = []
    all_events = []
    all_shots = []
    all_pstats = []

    rows_list = [r for _, r in df_matches.iterrows()]
```

```
    with ThreadPoolExecutor(max_workers=8) as executor:
        future_to_row = {executor.submit(fetch_single_match_payload, r): r for r in rows_list}
        done_count = 0
        for future in as_completed(future_to_row):
            r = future_to_row[future]
            done_count += 1
            match_id, payload = future.result()
            if payload:
                meta, st_r, ev_r, sh_r, pl_r = extract_from_match_payload(match_id, payload, r)
                all_meta.append(meta)
                all_stats.extend(st_r)
                all_events.extend(ev_r)
                all_shots.extend(sh_r)
                all_pstats.extend(pl_r)
                print(f"  [{done_count:02d}/52] ✅ {r['home_team']} vs {r['away_team']} ({r['score_str']}) ->
{len(sh_r)} tirs, {len(ev_r)} événements, {len(pl_r)} joueurs")
            else:
                print(f"  [{done_count:02d}/52] ⚠️ {r['home_team']} vs {r['away_team']} (payload non
disponible)")

    conn = sqlite3.connect(DB_PATH)

    def save_table(records, csv_name, sql_name):
        if not records:
            return
        df = pd.DataFrame(records)
        cpath = os.path.join(RAW_DIR, csv_name)
        df.to_csv(cpath, index=False, encoding="utf-8")
        df.to_sql(sql_name, conn, if_exists="replace", index=False)
        print(f"💾 {sql_name:<30} : {len(df):>5} lignes -> {cpath}")

    print("\n=== BILAN DES FICHIERS GÉNÉRÉS & TABLES SQLITE ===")
    save_table(all_meta, "fotmob_matches_metadata.csv", "fotmob_matches_metadata")
    save_table(all_stats, "fotmob_match_stats_can2025.csv", "fotmob_match_stats")
    save_table(all_events, "fotmob_match_events_can2025.csv", "fotmob_match_events")
    save_table(all_shots, "fotmob_shotmap_can2025.csv", "fotmob_shotmap")
    save_table(all_pstats, "fotmob_player_match_stats.csv", "fotmob_player_match_stats")

    conn.close()
    print("\n🎉 Extraction granulaire des 52 matchs terminée avec succès !")


if __name__ == "__main__":
    main()
```

4. Étape 3 : Collecte Complémentaire via API-Football & StatsBomb Open Data

En complément de FotMob, le script `scripts/03_fetch_real_data.py` montre comment interroger une API REST avec en-têtes d'authentification (`x-apisports-key` sur **API-Football**, `league=6`) ainsi que le dépôt JSON ouvert de **StatsBomb Open Data** (`competition_id=1267`).

Code Source Complet : `scripts/03_fetch_real_data.py`

```
# -*- coding: utf-8 -*-
"""
Étape Bonus Portfolio : Collecte Automatique de Données Réelles (API REST & Open Data)
Fichier : scripts/03_fetch_real_data.py
Auteur  : Amine

Ce script démontre comment un Data Analyst collecte des données depuis le Web via :
1. API-Football (REST API officielle avec clé API : équipes, matchs, joueurs CAN 2025)
2. StatsBomb Open Data API (100% Gratuit sans clé : événements détaillés de la CAN)
3. Sauvegarde automatique en CSV (data/raw/) et dans la base SQLite (data/can2025.db)
"""

import os
import sys
import json
import sqlite3
import urllib.request
import urllib.error
import pandas as pd

if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
RAW_DIR = os.path.join(BASE_DIR, "data", "raw")
DB_PATH = os.path.join(BASE_DIR, "data", "can2025.db")

# ==============================================================================
# CONFIGURATION : CLÉ API-FOOTBALL (Optionnelle)
# Créez un compte gratuit sur https://dashboard.api-football.com (100 req/jour)
# ==============================================================================
API_FOOTBALL_KEY = os.environ.get("API_FOOTBALL_KEY", "VOTRE_CLE_API_ICI")
AFCON_LEAGUE_ID = 6      # ID officiel de la Coupe d'Afrique des Nations dans API-Football
AFCON_SEASON = 2025      # Édition Maroc 2025


def fetch_json_url(url, headers=None):
    """Fonction utilitaire pour interroger une URL JSON sans dépendance externe."""
    req = urllib.request.Request(url, headers=headers or {"User-Agent": "Mozilla/5.0"})
    with urllib.request.urlopen(req, timeout=15) as response:
        return json.loads(response.read().decode("utf-8"))


# ==============================================================================
# SOURCE 1 : API-FOOTBALL (Calendrier, Scores, Stades & Joueurs CAN 2025)
# ==============================================================================
def fetch_from_api_football(api_key, league_id=AFCON_LEAGUE_ID, season=AFCON_SEASON):
    """
    Récupère tous les matchs et les meilleurs joueurs de la CAN depuis API-Football.
```

```
    """
    if not api_key or api_key == "VOTRE_CLE_API_ICI":
        print("ℹ️ [API-Football] Aucune clé API renseignée dans API_FOOTBALL_KEY.")
        print("   -> Pour activer API-Football : inscrivez-vous gratuitement sur https://dashboard.api-
football.com")
        print("   -> Puis définissez votre clé : set API_FOOTBALL_KEY=votre_cle dans le terminal.\n")
        return None, None

    headers = {
        "x-apisports-key": api_key,
        "Accept": "application/json"
    }

    print(f"🌐 [API-Football] Téléchargement des matchs (League={league_id}, Season={season})...")
    url_fixtures = f"https://v3.football.api-sports.io/fixtures?league={league_id}&season={season}"
    res_fixtures = fetch_json_url(url_fixtures, headers=headers)

    matches_list = []
    for item in res_fixtures.get("response", []):
        fix = item.get("fixture", {})
        venue = fix.get("venue", {})
        league = item.get("league", {})
        teams = item.get("teams", {})
        goals = item.get("goals", {})

        matches_list.append({
            "api_fixture_id": fix.get("id"),
            "match_date": (fix.get("date") or "")[:10],
            "stage": league.get("round"),
            "stadium": venue.get("name"),
            "host_city": venue.get("city"),
            "home_team": teams.get("home", {}).get("name"),
            "away_team": teams.get("away", {}).get("name"),
            "home_goals": goals.get("home"),
            "away_goals": goals.get("away"),
            "status": fix.get("status", {}).get("short")
        })

    df_api_matches = pd.DataFrame(matches_list)

    print(f"🌐 [API-Football] Téléchargement des meilleurs buteurs (Top Scorers)...")
    url_scorers = f"https://v3.football.api-sports.io/players/topscorers?league={league_id}&season={season}"
    res_scorers = fetch_json_url(url_scorers, headers=headers)

    scorers_list = []
    for item in res_scorers.get("response", []):
        player = item.get("player", {})
        stats = (item.get("statistics") or [{}])[0]
        team = stats.get("team", {})
        games = stats.get("games", {})
        goals = stats.get("goals", {})
        passes = stats.get("passes", {})
        dribbles = stats.get("dribbles", {})

        scorers_list.append({
            "player_id": player.get("id"),
            "player_name": player.get("name"),
            "age": player.get("age"),
            "nationality": player.get("nationality"),
            "team_name": team.get("name"),
            "position": games.get("position"),
            "matches_played": games.get("appearences"),
            "minutes_played": games.get("minutes"),
            "rating": float(games.get("rating") or 0.0),
            "goals": goals.get("total") or 0,
```

```
            "assists": goals.get("assists") or 0,
            "key_passes": passes.get("key") or 0,
            "dribbles_success": dribbles.get("success") or 0
        })

    df_api_scorers = pd.DataFrame(scorers_list)
    return df_api_matches, df_api_scorers


# ================================================================================
# SOURCE 2 : STATSBOMB OPEN DATA (100% GRATUIT - SANS CLÉ API)
# ================================================================================
def fetch_from_statsbomb_open_data():
    """
    Récupère gratuitement les matchs officiels de la CAN depuis le dépôt GitHub
    StatsBomb Open Data (competition_id = 1267 : African Cup of Nations).
    Aucune clé API n'est requise !
    """
    print("🌐 [StatsBomb Open Data] Connexion au dépôt GitHub officiel StatsBomb...")
    comp_url = "https://raw.githubusercontent.com/statsbomb/open-data/master/data/competitions.json"

    try:
        competitions = fetch_json_url(comp_url)
    except Exception as e:
        print(f"⚠️ Impossible de contacter GitHub StatsBomb ({e}). Vérifiez votre connexion Internet.")
        return None

    afcon_seasons = [
        c for c in competitions
        if "African Cup of Nations" in c.get("competition_name", "") or c.get("competition_id") == 1267
    ]

    if not afcon_seasons:
        print("ℹ️ Compétition CAN non trouvée dans la liste StatsBomb.")
        return None

    # On prend la saison la plus récente disponible dans StatsBomb Open Data
    latest = sorted(afcon_seasons, key=lambda x: x.get("season_name", ""), reverse=True)[0]
    comp_id = latest["competition_id"]
    season_id = latest["season_id"]
    season_name = latest["season_name"]

    print(f"✅ [StatsBomb] Édition trouvée : {latest['competition_name']} ({season_name}) [comp_id=
{comp_id}, season_id={season_id}]")

    matches_url = f"https://raw.githubusercontent.com/statsbomb/open-
data/master/data/matches/{comp_id}/{season_id}.json"
    matches_json = fetch_json_url(matches_url)

    rows = []
    for m in matches_json:
        rows.append({
            "statsbomb_match_id": m.get("match_id"),
            "match_date": m.get("match_date"),
            "stage": m.get("competition_stage", {}).get("name"),
            "home_team": m.get("home_team", {}).get("home_team_name"),
            "away_team": m.get("away_team", {}).get("away_team_name"),
            "home_score": m.get("home_score"),
            "away_score": m.get("away_score"),
            "stadium": (m.get("stadium") or {}).get("name"),
            "referee": (m.get("referee") or {}).get("name")
        })

    df_sb_matches = pd.DataFrame(rows).sort_values("match_date")
    print(f"✅ [StatsBomb] {len(df_sb_matches)} matchs officiels récupérés avec succès !")
```

```
    return df_sb_matches


# ================================================================================
# SAUVEGARDE DANS CSV (data/raw/) ET SQLITE (data/can2025.db)
# ================================================================================
def main():
    print("=" * 70)
    print("🚀 COLLECTE DE DONNÉES WEB & API — PROJET CAN 2025 (MAROC)")
    print("=" * 70 + "\n")

    conn = sqlite3.connect(DB_PATH)

    # 1. Test API-Football
    df_api_m, df_api_p = fetch_from_api_football(API_FOOTBALL_KEY)
    if df_api_m is not None and not df_api_m.empty:
        csv_m = os.path.join(RAW_DIR, "api_football_matches.csv")
        df_api_m.to_csv(csv_m, index=False, encoding="utf-8")
        df_api_m.to_sql("api_football_matches", conn, if_exists="replace", index=False)
        print(f"💾 Matchs API-Football sauvegardés dans : {csv_m} et table SQL 'api_football_matches'")

    if df_api_p is not None and not df_api_p.empty:
        csv_p = os.path.join(RAW_DIR, "api_football_topscorers.csv")
        df_api_p.to_csv(csv_p, index=False, encoding="utf-8")
        df_api_p.to_sql("api_football_topscorers", conn, if_exists="replace", index=False)
        print(f"💾 Buteurs API-Football sauvegardés dans : {csv_p} et table SQL 'api_football_topscorers'")

    # 2. Collecte gratuite StatsBomb Open Data
    df_sb = fetch_from_statsbomb_open_data()
    if df_sb is not None and not df_sb.empty:
        csv_sb = os.path.join(RAW_DIR, "statsbomb_afcon_matches.csv")
        df_sb.to_csv(csv_sb, index=False, encoding="utf-8")
        df_sb.to_sql("statsbomb_afcon_matches", conn, if_exists="replace", index=False)
        print(f"💾 Matchs StatsBomb sauvegardés dans : {csv_sb}")
        print("💾 Table SQL 'statsbomb_afcon_matches' ajoutée dans data/can2025.db !\n")
        print("Aperçu des 5 derniers matchs récupérés en ligne :")
        print(df_sb[["match_date", "stage", "home_team", "home_score", "away_score",
"away_team"]].tail(5).to_string(index=False))

    conn.close()
    print("\n✅ Collecte terminée avec succès !")


if __name__ == "__main__":
    main()
```

5. Étape 4 : Nettoyage des Données (ETL) & Modélisation SQL (SQLite)

A. Pipeline ETL Python (`scripts/01_clean_and_build_db.py`)

Ce script illustre le nettoyage des données brutes (suppression des espaces avec `.str.strip()`, dédoublonnage avec `.drop_duplicates()`, imputation des valeurs manquantes par la médiane et création des indicateurs par 90 minutes) avant de charger les tables dans `data/can2025.db`.

```
# -*- coding: utf-8 -*-
"""
Étape 1 & 2 du Projet Portfolio : Nettoyage des données (ETL Python) & Alimentation SQLite
Auteur : Amine
Projet : Data Analyse CAN 2025 (Maroc)
"""

import os
import sqlite3
import pandas as pd

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
RAW_DIR = os.path.join(BASE_DIR, "data", "raw")
PROC_DIR = os.path.join(BASE_DIR, "data", "processed")
DB_PATH = os.path.join(BASE_DIR, "data", "can2025.db")

def run_etl():
    print("=== 1. CHARGEMENT DES DONNÉES BRUTES ===")
    df_hist = pd.read_csv(os.path.join(RAW_DIR, "historical_afcon.csv"))
    df_teams = pd.read_csv(os.path.join(RAW_DIR, "teams_can2025.csv"))
    df_matches = pd.read_csv(os.path.join(RAW_DIR, "matches_can2025.csv"))
    df_players = pd.read_csv(os.path.join(RAW_DIR, "players_can2025.csv"))

    print(f"Équipes (brut) : {len(df_teams)} lignes")
    print(f"Matchs (brut)  : {len(df_matches)} lignes (Valeurs manquantes attendance :
{df_matches['attendance'].isna().sum()})")

    print("\n=== 2. NETTOYAGE DES DONNÉES (DATA CLEANING) ===")
    # A. Suppression des espaces superflus et des doublons dans teams
    df_teams["team_name"] = df_teams["team_name"].str.strip()
    df_teams = df_teams.drop_duplicates(subset=["team_id"], keep="first")
    print(f"Équipes (après suppression des doublons et nettoyage texte) : {len(df_teams)} lignes")

    # B. Imputation des valeurs manquantes d'affluence par la médiane de la ville hôte
    df_matches["attendance"] = df_matches.groupby("host_city")["attendance"].transform(
        lambda x: x.fillna(x.median())
    ).astype(int)

    # C. Feature Engineering sur les joueurs (Normalisation par 90 minutes)
    df_players["goals_per_90"] = ((df_players["goals"] / df_players["minutes_played"]) * 90).round(2)
    df_players["assists_per_90"] = ((df_players["assists"] / df_players["minutes_played"]) * 90).round(2)
    df_players["goal_contributions"] = df_players["goals"] + df_players["assists"]
    df_players["goal_contributions_per_90"] = ((df_players["goal_contributions"] /
df_players["minutes_played"]) * 90).round(2)
    df_players["xg_diff"] = (df_players["goals"] - df_players["xg"]).round(2)

    print("\n=== 3. EXPORT DES FICHIERS PROPRES (POUR EXCEL & POWER BI / TABLEAU) ===")
    df_hist.to_csv(os.path.join(PROC_DIR, "historical_afcon_clean.csv"), index=False, encoding="utf-8")
    df_teams.to_csv(os.path.join(PROC_DIR, "teams_can2025_clean.csv"), index=False, encoding="utf-8")
    df_matches.to_csv(os.path.join(PROC_DIR, "matches_can2025_clean.csv"), index=False, encoding="utf-8")
    df_players.to_csv(os.path.join(PROC_DIR, "players_can2025_clean.csv"), index=False, encoding="utf-8")
```

```
    print("\n=== 4. MISE À JOUR DE LA BASE DE DONNÉES SQLITE ===")
    conn = sqlite3.connect(DB_PATH)
    df_hist.to_sql("historical_afcon", conn, if_exists="replace", index=False)
    df_teams.to_sql("teams", conn, if_exists="replace", index=False)
    df_matches.to_sql("matches", conn, if_exists="replace", index=False)
    df_players.to_sql("players", conn, if_exists="replace", index=False)

    # Exemple d'exécution d'une requête SQL sur le Maroc
    query_maroc = """
    SELECT player_name, position, club, goals, assists, key_passes, rating
    FROM players
    WHERE team_id = 'MAR'
    ORDER BY rating DESC
    LIMIT 5;
    """
    print("\nTop 5 Joueurs du Maroc (via SQL) :")
    print(pd.read_sql_query(query_maroc, conn).to_string(index=False))
    conn.close()
    print("\nPipeline ETL terminé avec succès !")

if __name__ == "__main__":
    run_etl()
```

B. Schéma Relationnel SQL (`sql/schema.sql`)

```
-- ============================================================================
-- SCHEMA RELATIONNEL : BASE DE DONNÉES CAN 2025 (MAROC)
-- ============================================================================

DROP TABLE IF EXISTS players;
DROP TABLE IF EXISTS matches;
DROP TABLE IF EXISTS teams;
DROP TABLE IF EXISTS historical_afcon;

CREATE TABLE historical_afcon (
    year INTEGER PRIMARY KEY,
    host_country TEXT NOT NULL,
    winner TEXT NOT NULL,
    runner_up TEXT NOT NULL,
    num_teams INTEGER,
    matches_played INTEGER,
    total_goals INTEGER,
    goals_per_match REAL,
    avg_attendance INTEGER,
    host_won INTEGER
);

CREATE TABLE teams (
    team_id TEXT PRIMARY KEY,
    team_name TEXT NOT NULL,
    group_name TEXT NOT NULL,
    fifa_rank INTEGER,
    market_value_m_eur REAL,
    coach_style TEXT,
    confederation_region TEXT,
    previous_titles INTEGER,
    final_stage_2025 TEXT
);

CREATE TABLE matches (
    match_id TEXT PRIMARY KEY,
    match_date TEXT NOT NULL,
```

```
    stage TEXT NOT NULL,
    group_name TEXT,
    home_team_id TEXT NOT NULL,
    away_team_id TEXT NOT NULL,
    home_goals INTEGER,
    away_goals INTEGER,
    home_xg REAL,
    away_xg REAL,
    home_possession INTEGER,
    away_possession INTEGER,
    home_shots INTEGER,
    home_shots_on_target INTEGER,
    away_shots INTEGER,
    away_shots_on_target INTEGER,
    host_city TEXT,
    attendance INTEGER,
    total_goals INTEGER,
    FOREIGN KEY (home_team_id) REFERENCES teams(team_id),
    FOREIGN KEY (away_team_id) REFERENCES teams(team_id)
);

CREATE TABLE players (
    player_id TEXT PRIMARY KEY,
    player_name TEXT NOT NULL,
    team_id TEXT NOT NULL,
    team_name TEXT NOT NULL,
    position TEXT NOT NULL,
    age INTEGER,
    club TEXT,
    market_value_m_eur REAL,
    matches_played INTEGER,
    minutes_played INTEGER,
    goals INTEGER,
    xg REAL,
    assists INTEGER,
    xa REAL,
    key_passes INTEGER,
    progressive_carries INTEGER,
    tackles_won INTEGER,
    interceptions INTEGER,
    duels_won_pct REAL,
    clean_sheets INTEGER,
    rating REAL,
    goals_per_90 REAL,
    assists_per_90 REAL,
    goal_contributions INTEGER,
    goal_contributions_per_90 REAL,
    xg_diff REAL,
    FOREIGN KEY (team_id) REFERENCES teams(team_id)
);
```

C. Les Requêtes Analytiques SQL du Portfolio (`sql/business_queries.sql`)

```
-- ============================================================================
-- 12 REQUÊTES SQL ANALYTIQUES POUR LE PORTFOLIO DATA ANALYST (CAN 2025 MAROC)
-- Compétences : JOIN, GROUP BY, CTE (WITH), CASE WHEN, Window Functions (RANK)
-- ============================================================================

-- ----------------------------------------------------------------------------
-- VOLET 1 : HISTORIQUE & ÉVOLUTION DE LA CAN (1957 - 2025)
-- ----------------------------------------------------------------------------

-- Q1. Évolution de la moyenne de buts et de l'affluence sur les 10 dernières éditions
```

```sql
SELECT
    year AS edition,
    host_country AS pays_hote,
    winner AS vainqueur,
    matches_played AS matchs,
    total_goals AS buts_totaux,
    ROUND(goals_per_match, 2) AS buts_par_match,
    avg_attendance AS affluence_moyenne,
    CASE WHEN host_won = 1 THEN 'Oui' ELSE 'Non' END AS victoire_pays_hote
FROM historical_afcon
ORDER BY year DESC
LIMIT 10;

-- Q2. Taux de réussite historique des pays organisateurs de la CAN
SELECT
    COUNT(*) AS total_editions,
    SUM(host_won) AS titres_pays_hote,
    ROUND(100.0 * SUM(host_won) / COUNT(*), 1) AS pourcentage_victoire_hote
FROM historical_afcon;



-- ------------------------------------------------------------------------
-- VOLET 2 : BILAN COLLECTIF DES ÉQUIPES (CAN 2025)
-- ------------------------------------------------------------------------

-- Q3. Bilan complet par équipe (Matchs, Buts marqués, Buts encaissés, Différence, xG) via CTE
WITH team_matches AS (
    SELECT
        home_team_id AS team_id,
        home_goals AS goals_for,
        away_goals AS goals_against,
        home_xg AS xg_for,
        away_xg AS xg_against,
        home_possession AS possession
    FROM matches
    UNION ALL
    SELECT
        away_team_id AS team_id,
        away_goals AS goals_for,
        home_goals AS goals_against,
        away_xg AS xg_for,
        home_xg AS xg_against,
        away_possession AS possession
    FROM matches
)
SELECT
    t.team_name AS equipe,
    t.final_stage_2025 AS parcours,
    COUNT(*) AS matchs_joues,
    SUM(tm.goals_for) AS buts_marques,
    SUM(tm.goals_against) AS buts_encaisses,
    SUM(tm.goals_for) - SUM(tm.goals_against) AS diff_buts,
    ROUND(SUM(tm.xg_for), 2) AS xg_total,
    ROUND(SUM(tm.goals_for) - SUM(tm.xg_for), 2) AS surperformance_xg,
    ROUND(AVG(tm.possession), 1) AS possession_moyenne
FROM team_matches tm
JOIN teams t ON tm.team_id = t.team_id
GROUP BY t.team_id, t.team_name, t.final_stage_2025
ORDER BY buts_marques DESC, diff_buts DESC;

-- Q4. Affluence et spectacle par ville hôte marocaine
SELECT
    host_city AS ville_hote,
    COUNT(*) AS nb_matchs,
    SUM(total_goals) AS total_buts,
```

```sql
    ROUND(AVG(total_goals), 2) AS moyenne_buts_match,
    ROUND(AVG(attendance), 0) AS affluence_moyenne,
    MAX(attendance) AS affluence_record
FROM matches
GROUP BY host_city
ORDER BY affluence_moyenne DESC;


-- ------------------------------------------------------------------------
-- VOLET 3 : SCOUTING & RECRUTEMENT DE JOUEURS
-- ------------------------------------------------------------------------

-- Q5. Classement des meilleurs joueurs décisifs (Buts + Passes D) avec Window Function (DENSE_RANK)
SELECT
    DENSE_RANK() OVER (ORDER BY goal_contributions DESC, goals DESC, minutes_played ASC) AS rang,
    player_name AS joueur,
    team_name AS selection,
    position,
    club,
    goals AS buts,
    assists AS passes_decisives,
    goal_contributions AS total_decisif,
    goal_contributions_per_90 AS decisif_par_90min,
    rating AS note_moyenne
FROM players
ORDER BY rang ASC
LIMIT 10;

-- Q6. Détection des "Pépites U23 sous-cotées" (Âge <= 23 ans, Valeur <= 20 M€, Haute performance)
SELECT
    player_name AS pepite_u23,
    team_name AS selection,
    age,
    position,
    club,
    market_value_m_eur AS valeur_marchande_meur,
    goals AS buts,
    assists AS passes_d,
    key_passes AS passes_cles,
    progressive_carries AS progressions_balle,
    rating AS note_tournoi
FROM players
WHERE age <= 23
  AND market_value_m_eur <= 20.0
  AND minutes_played >= 300
ORDER BY rating DESC;

-- Q7. Meilleur joueur par poste (Gardien, Défenseur, Milieu, Attaquant) avec ROW_NUMBER()
WITH ranked_by_pos AS (
    SELECT
        position,
        player_name,
        team_name,
        club,
        goals,
        assists,
        tackles_won,
        interceptions,
        duels_won_pct,
        rating,
        ROW_NUMBER() OVER (PARTITION BY position ORDER BY rating DESC) AS pos_rank
    FROM players
)
SELECT
    position,
```

```sql
    player_name AS meilleur_joueur,
    team_name AS selection,
    club,
    rating AS note_moyenne,
    duels_won_pct AS pct_duels_gagnes
FROM ranked_by_pos
WHERE pos_rank <= 2
ORDER BY position, pos_rank;


-- ------------------------------------------------------------------------
-- VOLET 4 : FOCUS SPÉCIAL MAROC (LIONS DE L'ATLAS)
-- ------------------------------------------------------------------------

-- Q8. Parcours match par match du Maroc à la CAN 2025
SELECT
    m.match_id,
    m.match_date AS date_match,
    m.stage AS tour,
    t_opp.team_name AS adversaire,
    CASE WHEN m.home_team_id = 'MAR' THEN m.home_goals ELSE m.away_goals END AS buts_maroc,
    CASE WHEN m.home_team_id = 'MAR' THEN m.away_goals ELSE m.home_goals END AS buts_adversaire,
    CASE WHEN m.home_team_id = 'MAR' THEN m.home_xg ELSE m.away_xg END AS xg_maroc,
    CASE WHEN m.home_team_id = 'MAR' THEN m. away_xg ELSE m.home_xg END AS xg_adversaire,
    CASE WHEN m.home_team_id = 'MAR' THEN m.home_possession ELSE m.away_possession END AS possession_maroc,
    m.host_city AS ville,
    m.attendance AS spectateurs
FROM matches m
JOIN teams t_opp ON (
    CASE WHEN m.home_team_id = 'MAR' THEN m.away_team_id ELSE m.home_team_id END = t_opp.team_id
)
WHERE m.home_team_id = 'MAR' OR m.away_team_id = 'MAR'
ORDER BY m.match_date ASC;

-- Q9. Contributions individuelles de l'effectif du Maroc
SELECT
    player_name AS joueur_marocain,
    position AS poste,
    club,
    minutes_played AS minutes,
    goals AS buts,
    assists AS passes_d,
    key_passes AS passes_cles,
    progressive_carries AS progressions,
    tackles_won + interceptions AS actions_defensives,
    duels_won_pct AS duels_gagnes_pct,
    rating AS note
FROM players
WHERE team_id = 'MAR'
ORDER BY rating DESC;
```

6. Étape 5 : Code de l'Application Web Interactive (Streamlit & Plotly)

Le fichier `app/streamlit_app.py` connecte la base SQLite `data/can2025.db` à une interface web interactive structurée en **6 onglets**, incluant le tracé géométrique d'un demi-terrain de football réglementaire (105x68) avec Plotly pour afficher les 1 026 tirs de la Shotmap FotMob.

Code Source Complet : `app/streamlit_app.py`

```
# -*- coding: utf-8 -*-
"""
Application Interactive de Data Analyse & Scouting : CAN 2025 (Maroc)
Intègre les données officielles + l'extraction granulaire FotMob (1 026 tirs, 710 événements, 2 445
prestations joueurs)
Lancez avec : python -m streamlit run app/streamlit_app.py
"""

import os
import sqlite3
import pandas as pd
import streamlit as st
import plotly.express as px
import plotly.graph_objects as go

st.set_page_config(
    page_title="CAN 2025 Maroc - Data Analyse, Shotmap & Match Center FotMob",
    page_icon="🏆",
    layout="wide"
)

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DB_PATH = os.path.join(BASE_DIR, "data", "can2025.db")


@st.cache_data
def load_all_data(db_mtime):
    conn = sqlite3.connect(DB_PATH)
    hist = pd.read_sql_query("SELECT * FROM historical_afcon ORDER BY year", conn)
    teams = pd.read_sql_query("SELECT * FROM teams", conn)
    matches = pd.read_sql_query("SELECT * FROM matches", conn)
    players = pd.read_sql_query("SELECT * FROM players", conn)

    # Tables FotMob extraites
    def safe_read_table(tname):
        try:
            return pd.read_sql_query(f"SELECT * FROM {tname}", conn)
        except Exception:
            return pd.DataFrame()

    fm_standings = safe_read_table("fotmob_standings")
    fm_pstats = safe_read_table("fotmob_player_stats")
    fm_meta = safe_read_table("fotmob_matches_metadata")
    fm_mstats = safe_read_table("fotmob_match_stats")
    fm_events = safe_read_table("fotmob_match_events")
    fm_shots = safe_read_table("fotmob_shotmap")
    fm_pmatch = safe_read_table("fotmob_player_match_stats")

    conn.close()

    # Associer le nom de l'équipe aux tirs FotMob via team_id
```

```
    if not fm_shots.empty and not fm_standings.empty:
        tid_map = dict(zip(fm_standings["team_id"].astype(str), fm_standings["team_name"]))
        fm_shots["team_name"] = fm_shots["team_id"].astype(str).map(tid_map).fillna("Autre")

    return hist, teams, matches, players, fm_standings, fm_pstats, fm_meta, fm_mstats, fm_events, fm_shots,
fm_pmatch


db_mtime = os.path.getmtime(DB_PATH) if os.path.exists(DB_PATH) else 0
(
    df_hist, df_teams, df_matches, df_players,
    df_fm_standings, df_fm_pstats, df_fm_meta,
    df_fm_mstats, df_fm_events, df_fm_shots, df_fm_pmatch
) = load_all_data(db_mtime)


def draw_half_pitch_shotmap(df_s, title="Carte des Tirs (Shotmap FotMob)"):
    """Dessine un demi-terrain de football réglementaire (dimensions Opta/FotMob 105x68) avec les tirs."""
    fig = go.Figure()

    # Couleur de fond terrain et lignes blanches (demi-terrain offensif : x de 52.5 à 106, y de -2 à 70)
    pitch_shapes = [
        # Pelouse
        dict(type="rect", x0=52.5, y0=0, x1=105, y1=68, line=dict(color="white", width=2),
fillcolor="#1e7b34", layer="below"),
        # Grande surface de réparation (16.5m : x de 88.5 à 105, y de 13.84 à 54.16)
        dict(type="rect", x0=88.5, y0=13.84, x1=105, y1=54.16, line=dict(color="white", width=2),
fillcolor="rgba(0,0,0,0)", layer="below"),
        # Surface de but (5.5m : x de 99.5 à 105, y de 24.84 à 43.16)
        dict(type="rect", x0=99.5, y0=24.84, x1=105, y1=43.16, line=dict(color="white", width=2),
fillcolor="rgba(0,0,0,0)", layer="below"),
        # Cages du but (y de 30.34 à 37.66)
        dict(type="rect", x0=105, y0=30.34, x1=106.2, y1=37.66, line=dict(color="#f1c40f", width=3),
fillcolor="rgba(255,255,255,0.25)", layer="below"),
        # Point de penalty (x=94, y=34)
        dict(type="circle", x0=93.6, y0=33.6, x1=94.4, y1=34.4, line=dict(color="white", width=2),
fillcolor="white", layer="below"),
    ]

    color_map = {
        "Goal": "#f1c40f",          # Or vif pour les buts
        "AttemptSaved": "#3498db",  # Bleu pour les tirs arrêtés/bloqués
        "Miss": "#e74c3c",          # Rouge pour les tirs hors cadre
        "Post": "#9b59b6"           # Violet pour les poteaux
    }
    symbol_map = {
        "Goal": "star",
        "AttemptSaved": "circle",
        "Miss": "x",
        "Post": "diamond"
    }

    for ev_type, group in df_s.groupby("event_type"):
        xg_vals = pd.to_numeric(group["xg"], errors="coerce").fillna(0.05)
        marker_sizes = (xg_vals * 38 + 9).clip(9, 32)
        fig.add_trace(go.Scatter(
            x=group["x"],
            y=group["y"],
            mode="markers",
            name=f"{ev_type} ({len(group)})",
            marker=dict(
                size=marker_sizes,
                color=color_map.get(ev_type, "#ecf0f1"),
                symbol=symbol_map.get(ev_type, "circle"),
                line=dict(width=1.2, color="black"),
```

```
                opacity=0.9
            ),
            customdata=group[["player_name", "team_name", "minute", "xg", "xgot", "shot_type", "situation",
"home_team", "away_team"]],
            hovertemplate=(
                "<b>%{customdata[0]}</b> (%{customdata[1]}) — %{customdata[2]}'<br>"
                "Match : %{customdata[7]} vs %{customdata[8]}<br>"
                "Résultat : <b>" + str(ev_type) + "</b> | Type : %{customdata[5]} (%{customdata[6]})<br>"
                "xG : <b>%{customdata[3]:.2f}</b> | xGOT : %{customdata[4]:.2f}<extra></extra>"
            )
        ))

    fig.update_layout(
        title=title,
        shapes=pitch_shapes,
        xaxis=dict(range=[52, 107], showgrid=False, zeroline=False, visible=False),
        yaxis=dict(range=[-2, 70], showgrid=False, zeroline=False, visible=False, scaleanchor="x",
scaleratio=1),
        plot_bgcolor="#145a23",
        height=540,
        margin=dict(l=10, r=10, t=45, b=10),
        legend=dict(orientation="h", yanchor="bottom", y=1.01, xanchor="right", x=1)
    )
    return fig


# ================================================================================
# EN-TÊTE GLOBAL & KPIS OFFICIELS DU TOURNOI (CAN 2025 MAROC)
# ================================================================================
st.title("🏆 CAN 2025 (Maroc) — Plateforme Data Analyse, Shotmap & Match Center FotMob")
st.markdown(
    "**Données Officielles & Extraction Temps Réel FotMob** | **52 matchs**, **1 026 tirs géolocalisés
(xG)**, "
    "**710 événements minute par minute** et **2 445 prestations individuelles de joueurs**."
)

c1, c2, c3, c4, c5 = st.columns(5)
c1.metric("Matchs & Tirs FotMob", f"52 matchs | {len(df_fm_shots)} tirs", f"{len(df_fm_pmatch)} fiches
joueurs")
c2.metric("Buts Marqués", int(df_matches["total_goals"].sum()), f"Moyenne :
{df_matches['total_goals'].mean():.2f} / match")
c3.metric("Soulier d'Or (Buteur)", "Brahim Díaz MA", "5 buts (Osimhen & Salah : 4)")
c4.metric("Meilleur Joueur & Gardien", "Mané SN / Bounou MA", "Lookman NG : 4 passes D")
c5.metric("Champion CAN 2025", "MA Maroc (3-0 CAF)", "Finaliste : Sénégal SN")

st.divider()

tab1, tab2, tab3, tab4, tab5, tab6 = st.tabs([
    "🎯 1. Diagnostic par Équipe (24 Nations)",
    "🥅 2. Shotmap Interactive FotMob (1 026 Tirs)",
    "🏟️ 3. Feuille de Match FotMob (Match Center)",
    "📊 4. Bilan des 24 Équipes & Groupes",
    "🔍 5. Scouting & Stats Joueurs FotMob",
    "📜 6. Historique CAN (1957-2025)"
])

# ================================================================================
# ONGLET 1 : DIAGNOSTIC TACTIQUE & PARCOURS DE CHAQUE ÉQUIPE (LISTE DÉROULANTE)
# ================================================================================
with tab1:
    st.subheader("🎯 Diagnostic Tactique & Parcours Officiel par Équipe Nationale")

    teams_sorted = df_teams.sort_values(["group_name", "fifa_rank"]).copy()
    team_options = [
        f"{r['team_name']} ({r['team_id']}) — {r['group_name']} | {r['final_stage_2025']}"
```

```
        for _, r in teams_sorted.iterrows()
    ]
    default_idx = next((i for i, opt in enumerate(team_options) if "(MAR)" in opt), 0)

    col_sel, col_info = st.columns([1.35, 2])
    with col_sel:
        selected_label = st.selectbox(
            "🔽 Choisissez une équipe parmi les 24 nations qualifiées :",
            options=team_options,
            index=default_idx
        )
    selected_team_id = selected_label.split("(")[1].split(")")[0]
    team_info = df_teams[df_teams["team_id"] == selected_team_id].iloc[0]

    with col_info:
        st.info(
            f"**{team_info['team_name']}** | **Groupe :** {team_info['group_name']} | "
            f"**Parcours réel :** *{team_info['final_stage_2025']}* | "
            f"**Style Tactique :** *{team_info['coach_style']}* | "
            f"**Titres CAN avant 2025 :** {team_info['previous_titles']}"
        )

    raw_tm = df_matches[
        (df_matches["home_team_id"] == selected_team_id) |
        (df_matches["away_team_id"] == selected_team_id)
    ].sort_values("match_date").copy()

    team_name_map = dict(zip(df_teams["team_id"], df_teams["team_name"]))

    match_records = []
    wins, draws, losses = 0, 0, 0
    for _, m in raw_tm.iterrows():
        is_home = (m["home_team_id"] == selected_team_id)
        opp_id = m["away_team_id"] if is_home else m["home_team_id"]
        opp_name = team_name_map.get(opp_id, opp_id)
        gf = int(m["home_goals"] if is_home else m["away_goals"])
        ga = int(m["away_goals"] if is_home else m["home_goals"])
        xgf = float(m["home_xg"] if is_home else m["away_xg"])
        xga = float(m["away_xg"] if is_home else m["home_xg"])
        poss = int(m["home_possession"] if is_home else m["away_possession"])
        shots = int(m["home_shots"] if is_home else m["away_shots"])
        sot = int(m["home_shots_on_target"] if is_home else m["away_shots_on_target"])
        stage_str = m["stage"]

        if gf > ga:
            res_label = "✅ Victoire"
            wins += 1
        elif gf == ga:
            draws += 1
            if f"tab {selected_team_id}" in stage_str:
                res_label = "✅ Qualifié (t.a.b.)"
            elif "tab" in stage_str:
                res_label = "❌ Éliminé (t.a.b.)"
            else:
                res_label = "🤝 Match Nul"
        else:
            losses += 1
            res_label = "❌ Défaite"

        match_records.append({
            "Match ID": m["match_id"],
            "Date": m["match_date"],
            "Tour": stage_str,
            "Adversaire": opp_name,
            "Résultat": res_label,
```

```
            "Score": f"{gf} - {ga}",
            "Buts Pour": gf,
            "Buts Contre": ga,
            "xG Créés": round(xgf, 2),
            "xG Concédés": round(xga, 2),
            "Possession (%)": poss,
            "Tirs (Cadrés)": f"{shots} ({sot})",
            "Ville Hôte": m["host_city"],
            "Affluence": int(m["attendance"]),
            "Label_Graph": f"vs {opp_name} ({gf}-{ga})"
        })

    df_tm_view = pd.DataFrame(match_records)

    k1, k2, k3, k4, k5, k6 = st.columns(6)
    k1.metric("Parcours CAN 2025", team_info["final_stage_2025"])
    k2.metric("Bilan", f"{wins}V - {draws}N - {losses}D", f"{len(df_tm_view)} matchs joués")
    total_gf = int(df_tm_view["Buts Pour"].sum())
    total_ga = int(df_tm_view["Buts Contre"].sum())
    k3.metric("Buts Marqués / Encaissés", f"{total_gf} BP / {total_ga} BC", f"Diff : {total_gf -
total_ga:+d}")
    tot_xgf = df_tm_view["xG Créés"].sum()
    tot_xga = df_tm_view["xG Concédés"].sum()
    k4.metric("Total Expected Goals (xG)", f"{tot_xgf:.2f} xG", f"xG Concédés : {tot_xga:.2f}")
    k5.metric("Possession Moyenne", f"{df_tm_view['Possession (%)'].mean():.1f}%")
    k6.metric("Classement FIFA & Valeur", f"#{team_info['fifa_rank']} FIFA", f"
{team_info['market_value_m_eur']:.1f} M€")

    g_col1, g_col2 = st.columns([1.15, 1])

    with g_col1:
        fig_xg = go.Figure()
        fig_xg.add_trace(go.Bar(
            x=df_tm_view["Label_Graph"],
            y=df_tm_view["xG Créés"],
            name=f"xG Créés ({team_info['team_name']})",
            marker_color="#c1272d",
            text=df_tm_view["xG Créés"],
            textposition="auto"
        ))
        fig_xg.add_trace(go.Bar(
            x=df_tm_view["Label_Graph"],
            y=df_tm_view["xG Concédés"],
            name="xG Concédés (Adversaire)",
            marker_color="#7f8c8d",
            text=df_tm_view["xG Concédés"],
            textposition="auto"
        ))
        fig_xg.update_layout(
            barmode="group",
            title=f"Parcours Match par Match : xG de {team_info['team_name']} vs Adversaires",
            xaxis_title="Matchs disputés dans le tournoi",
            yaxis_title="Expected Goals (xG)"
        )
        st.plotly_chart(fig_xg, use_container_width=True)

    with g_col2:
        team_players = df_players[df_players["team_id"] == selected_team_id].sort_values(
            ["goal_contributions", "rating"], ascending=False
        )
        fig_pl = px.bar(
            team_players.head(10),
            x="rating",
            y="player_name",
            color="position",
```

```
            orientation="h",
            hover_data=["club", "goals", "assists", "key_passes", "duels_won_pct"],
            title=f"Joueurs Clés de {team_info['team_name']} (Classés par Note & Impact)",
            labels={"rating": "Note Moyenne (/10)", "player_name": "Joueur", "position": "Poste"}
        )
        fig_pl.update_layout(yaxis={"categoryorder": "total ascending"}, xaxis_range=[6.0, 9.0])
        st.plotly_chart(fig_pl, use_container_width=True)

    st.markdown(f"#### 🗓️ Calendrier & Résultats Officiels — {team_info['team_name']}")
    st.dataframe(
        df_tm_view[[
            "Date", "Tour", "Adversaire", "Résultat", "Score",
            "xG Créés", "xG Concédés", "Possession (%)", "Tirs (Cadrés)", "Ville Hôte", "Affluence"
        ]],
        use_container_width=True,
        hide_index=True
    )

    st.markdown(f"#### 👥 Statistiques Individuelles des Joueurs — {team_info['team_name']}")
    st.dataframe(
        team_players[[
            "player_name", "position", "age", "club", "market_value_m_eur",
            "matches_played", "minutes_played", "goals", "xg", "assists",
            "key_passes", "progressive_carries", "tackles_won", "interceptions",
            "duels_won_pct", "clean_sheets", "rating"
        ]].sort_values("rating", ascending=False),
        use_container_width=True,
        hide_index=True
    )

# ==============================================================================
# ONGLET 2 : CARTE INTERACTIVE DES TIRS (SHOTMAP FOTMOB - 1 026 TIRS RÉELS)
# ==============================================================================
with tab2:
    st.subheader("🥅 Carte Interactive des Tirs (Shotmap Officielle FotMob — 1 026 Tirs)")
    if df_fm_shots.empty:
        st.warning("Aucune donnée Shotmap FotMob trouvée.")
    else:
        sc1, sc2, sc3, sc4 = st.columns(4)
        all_shot_teams = ["Toutes les équipes"] + sorted(df_fm_shots["team_name"].dropna().unique().tolist())
        default_shot_team = all_shot_teams.index("Morocco") if "Morocco" in all_shot_teams else 0

        sel_shot_team = sc1.selectbox("Filtrer par Équipe (FotMob) :", options=all_shot_teams,
index=default_shot_team)

        sub_shots = df_fm_shots.copy()
        if sel_shot_team != "Toutes les équipes":
            sub_shots = sub_shots[sub_shots["team_name"] == sel_shot_team]

        all_shot_players = ["Tous les joueurs"] + sorted(sub_shots["player_name"].dropna().unique().tolist())
        sel_shot_player = sc2.selectbox("Filtrer par Joueur :", options=all_shot_players, index=0)
        if sel_shot_player != "Tous les joueurs":
            sub_shots = sub_shots[sub_shots["player_name"] == sel_shot_player]

        ev_types = sorted(df_fm_shots["event_type"].dropna().unique().tolist())
        sel_ev_types = sc3.multiselect("Résultat du tir :", options=ev_types, default=ev_types)
        if sel_ev_types:
            sub_shots = sub_shots[sub_shots["event_type"].isin(sel_ev_types)]

        sh_types = sorted(df_fm_shots["shot_type"].dropna().unique().tolist())
        sel_sh_types = sc4.multiselect("Partie du corps :", options=sh_types, default=sh_types)
        if sel_sh_types:
            sub_shots = sub_shots[sub_shots["shot_type"].isin(sel_sh_types)]

        # KPIs des tirs filtrés
```

```
        sk1, sk2, sk3, sk4, sk5 = st.columns(5)
        n_shots = len(sub_shots)
        n_goals = int((sub_shots["event_type"] == "Goal").sum())
        tot_xg_s = pd.to_numeric(sub_shots["xg"], errors="coerce").sum()
        tot_xgot_s = pd.to_numeric(sub_shots["xgot"], errors="coerce").sum()
        conv_rate = (100.0 * n_goals / n_shots) if n_shots > 0 else 0.0

        sk1.metric("Tirs Sélectionnés", n_shots)
        sk2.metric("Buts Marqués (Goals)", n_goals, f"Conversion : {conv_rate:.1f}%")
        sk3.metric("Expected Goals Cumulés (xG)", f"{tot_xg_s:.2f}")
        sk4.metric("xG Cadrés (xGOT)", f"{tot_xgot_s:.2f}")
        sk5.metric("xG Moyen / Tir", f"{(tot_xg_s / n_shots):.3f}" if n_shots > 0 else "0.000")

        col_pitch, col_table = st.columns([1.35, 1])
        with col_pitch:
            pitch_fig = draw_half_pitch_shotmap(
                sub_shots,
                title=f"Shotmap sur Demi-Terrain — {sel_shot_team} ({sel_shot_player})"
            )
            st.plotly_chart(pitch_fig, use_container_width=True)

        with col_table:
            st.markdown("#### 📋 Détail des Tirs Sélectionnés (Classés par xG)")
            st.dataframe(
                sub_shots[[
                    "match_date", "player_name", "team_name", "minute",
                    "event_type", "xg", "xgot", "shot_type", "situation", "home_team", "away_team"
                ]].sort_values("xg", ascending=False),
                use_container_width=True,
                hide_index=True,
                height=490
            )

# ================================================================================
# ONGLET 3 : EXPLORATEUR DE FEUILLE DE MATCH FOTMOB (MATCH CENTER)
# ================================================================================
with tab3:
    st.subheader("🏟️ Explorateur de Feuille de Match FotMob (Match Center — 52 Matchs)")
    if df_fm_meta.empty:
        st.warning("Aucune métadonnée de match FotMob trouvée.")
    else:
        meta_sorted = df_fm_meta.sort_values("match_date", ascending=False).copy()
        meta_sorted["label"] = (
            meta_sorted["match_date"].astype(str) + " | " +
            meta_sorted["home_team"].astype(str) + " " +
            meta_sorted["score_str"].astype(str) + " " +
            meta_sorted["away_team"].astype(str) + " (" +
            meta_sorted["stadium_name"].fillna("Stade Maroc").astype(str) + ") [ID:" +
            meta_sorted["fotmob_match_id"].astype(str) + "]"
        )
        # Par défaut sur un match du Maroc (ex: Finale ou Maroc vs Comores)
        mar_match_idx = next((i for i, lbl in enumerate(meta_sorted["label"]) if "Morocco" in lbl), 0)

        sel_match_lbl = st.selectbox(
            "🔽 Sélectionnez l'un des 52 matchs de la CAN 2025 pour inspecter sa Feuille de Match :",
            options=meta_sorted["label"].tolist(),
            index=mar_match_idx
        )
        sel_mid = sel_match_lbl.split("[ID:")[1].split("]")[0]
        m_row = meta_sorted[meta_sorted["fotmob_match_id"].astype(str) == str(sel_mid)].iloc[0]

        # Bandeau d'informations officielles du match
        mc1, mc2, mc3, mc4, mc5 = st.columns(5)
        mc1.metric("Affiche & Score", f"{m_row['home_team']} {m_row['score_str']} {m_row['away_team']}")
        mc2.metric("Stade & Ville", str(m_row["stadium_name"] or "Maroc"), str(m_row["city"] or ""))
```

```
        att_val = f"{int(float(m_row['attendance'])):,}".replace(",", " ") if pd.notna(m_row["attendance"])
and str(m_row["attendance"]) != "" else "N/A"
        mc3.metric("Affluence Officielle", f"{att_val} spectateurs")
        mc4.metric("Arbitre Officiel", str(m_row["referee"] or "CAF"))
        mc5.metric("🌟 Homme du Match (FotMob)", str(m_row["player_of_the_match"] or "N/A"))

        col_ev, col_st = st.columns([1.1, 1.1])

        with col_ev:
            st.markdown("#### 🕰️ Chronologie Minute par Minute (Buts, Cartons, Changements)")
            m_events = df_fm_events[df_fm_events["fotmob_match_id"].astype(str) ==
str(sel_mid)].sort_values("minute")
            if not m_events.empty:
                st.dataframe(
                    m_events[["minute", "team_side", "event_type", "player_name", "card_type", "detail"]],
                    use_container_width=True,
                    hide_index=True,
                    height=360
                )
            else:
                st.info("Aucun événement détaillé enregistré pour ce match.")

        with col_st:
            st.markdown("#### 📊 Face-à-Face Statistique (FotMob Match Stats)")
            m_stats = df_fm_mstats[df_fm_mstats["fotmob_match_id"].astype(str) == str(sel_mid)]
            if not m_stats.empty:
                st.dataframe(
                    m_stats[["stat_group", "stat_name", "home_value", "away_value"]].rename(
                        columns={
                            "stat_group": "Catégorie",
                            "stat_name": "Indicateur",
                            "home_value": str(m_row["home_team"]),
                            "away_value": str(m_row["away_team"])
                        }
                    ),
                    use_container_width=True,
                    hide_index=True,
                    height=360
                )
            else:
                st.info("Statistiques collectives non disponibles pour ce match.")

        # Shotmap spécifique au match + Notes FotMob des 22 joueurs du match
        col_mshot, col_mp = st.columns([1.2, 1.1])
        with col_mshot:
            m_shots = df_fm_shots[df_fm_shots["fotmob_match_id"].astype(str) == str(sel_mid)]
            if not m_shots.empty:
                st.plotly_chart(
                    draw_half_pitch_shotmap(
                        m_shots,
                        title=f"Shotmap du Match : {m_row['home_team']} vs {m_row['away_team']}
({len(m_shots)} tirs)"
                    ),
                    use_container_width=True
                )
            else:
                st.info("Pas de Shotmap pour ce match.")

        with col_mp:
            st.markdown("#### 🌟 Notes FotMob & Prestations des Joueurs sur ce Match")
            m_players = df_fm_pmatch[df_fm_pmatch["fotmob_match_id"].astype(str) == str(sel_mid)].copy()
            if not m_players.empty:
                m_players["fotmob_rating_num"] = pd.to_numeric(m_players["fotmob_rating"], errors="coerce")
                m_players = m_players.sort_values("fotmob_rating_num", ascending=False)
                st.dataframe(
```

```
                    m_players[[
                        "player_name", "team_name", "fotmob_rating", "minutes_played",
                        "goals", "assists", "xg", "xa", "total_shots", "chances_created", "touches",
"duels_won"
                    ]],
                    use_container_width=True,
                    hide_index=True,
                    height=490
                )
            else:
                st.info("Notes individuelles non disponibles pour ce match.")

# ================================================================================
# ONGLET 4 : BILAN COMPARATIF DES 24 ÉQUIPES & GROUPES FOTMOB
# ================================================================================
with tab4:
    st.subheader("📊 Comparaison Collective des 24 Sélections & Classements FotMob")
    fig_val = px.scatter(
        df_teams,
        x="fifa_rank",
        y="market_value_m_eur",
        color="final_stage_2025",
        size="market_value_m_eur",
        hover_name="team_name",
        text="team_name",
        title="Classement FIFA vs Valeur Marchande et Parcours Réel à la CAN 2025",
        labels={"fifa_rank": "Classement FIFA (plus petit = meilleur)", "market_value_m_eur": "Valeur
Marchande (M€)"}
    )
    fig_val.update_traces(textposition="top center")
    st.plotly_chart(fig_val, use_container_width=True)

    if not df_fm_standings.empty:
        st.markdown("#### 🏆 Classements Officiels des Groupes extraits de FotMob")
        st.dataframe(df_fm_standings, use_container_width=True, hide_index=True)

# ================================================================================
# ONGLET 5 : SCOUTING & CLASSEMENTS OFFICIELS FOTMOB (123 STATS JOUEURS)
# ================================================================================
with tab5:
    st.subheader("🔍 Cellule de Recrutement & Top Performers FotMob")
    if not df_fm_pstats.empty:
        cats = df_fm_pstats["category"].unique().tolist()
        sel_cat = st.selectbox("Choisissez une catégorie statistique FotMob :", options=cats, index=0)
        sub_cat = df_fm_pstats[df_fm_pstats["category"] == sel_cat].sort_values("rank")
        fig_cat = px.bar(
            sub_cat.head(12),
            x="stat_value",
            y="name",
            color="team_name",
            orientation="h",
            title=f"Classement Officiel FotMob – {sel_cat}",
            labels={"stat_value": sel_cat, "name": "Joueur", "team_name": "Sélection"}
        )
        fig_cat.update_layout(yaxis={"categoryorder": "total ascending"})
        st.plotly_chart(fig_cat, use_container_width=True)

    st.markdown("#### 💎 Table Complète de Scouting (Valeur Marchande, Âge, Stats & Notes)")
    st.dataframe(df_players.sort_values(["goals", "assists", "rating"], ascending=False),
use_container_width=True, hide_index=True)

# ================================================================================
# ONGLET 6 : HISTORIQUE DE LA CAN (1957 - 2025)
# ================================================================================
with tab6:
```

```
    st.subheader("📜 Évolution Historique de la CAN (1957 - 2025)")
    fig_h = px.line(
        df_hist,
        x="year",
        y="goals_per_match",
        markers=True,
        hover_data=["host_country", "winner", "avg_attendance"],
        title="Évolution de la Moyenne de Buts par Match de 1957 à 2025"
    )
    st.plotly_chart(fig_h, use_container_width=True)
    st.dataframe(df_hist.sort_values("year", ascending=False), use_container_width=True, hide_index=True)
```

🚀 **Commandes récapitulatives pour relancer tout le projet à tout moment :**

1. Mettre à jour l'extraction globale FotMob : `python scripts/04_scrape_fotmob_can2025.py`
2. Mettre à jour les 52 feuilles de match et la Shotmap : `python scripts/05_scrape_fotmob_match_details.py`
3. Générer les 5 graphiques PNG haute définition : `python scripts/02_generate_charts.py`
4. Lancer l'application interactive dans le navigateur : `python -m streamlit run app/streamlit_app.py`